Project Synopsis

Autonomous ML Model Monitoring & Drift Management Platform

Cover Information:

• **Project Title:** Autonomous ML Model Monitoring & Drift Management Platform
• **Project Type:** Mini Project
• **Team Name/ID:** Team 76
• **Course:** B. Tech CSE – AIML
• **Institute:** GLA University, Mathura
• **Date:** 08 Feb 2026

---

1. Overview

1.1 Problem Statement

Machine Learning models deployed in production environments are subject to performance degradation over time due to changes in data distribution, business environment, and user behavior. This degradation, known as **Model Drift**, can cause significant operational and financial losses.

Current monitoring approaches are either:

- Manual
- Periodic
- Reactive
- Dependent on human supervision

There is no fully autonomous system that detects drift and retrains models automatically in an academic-friendly scalable format.

1.2 Goal

To develop an enterprise-grade autonomous platform that:

- Continuously monitors ML models
- Detects data, concept, and prediction drift
- Automatically triggers retraining

- Deploys updated model versions
- Provides centralized dashboard monitoring

1.3 Non-Goals

- Reinforcement learning adaptive systems
- Federated distributed training
- Cross-region fault tolerance
- Automated compliance certification

1.4 Value Proposition

Feature	**Traditional Monitoring**	**Proposed Platform**
Drift Detection	Manual	Automated
Retraining	Human-triggered	Autonomous
Monitoring	Periodic	Continuous
Deployment	Manual	Version-controlled auto-deploy
Cost	High enterprise	Academic scalable

2. Scope and Control

2.1 In-Scope

- Model registration
- Data monitoring
- Statistical drift detection
- Retraining pipeline
- Alert system
- Dashboard

2.2 Out-of-Scope

- Multi-asset real-time GPU orchestration
- Blockchain audit storage
- Mobile application

2.3 Assumptions

Assumption	Description
Training Data Available	Historical baseline exists
Labels Accessible	For retraining validation
Compute Available	Cloud/Local server

Assumption	Description
API Integration Possible	For model access

2.4 Constraints

Constraint	Impact
Academic timeline	Limited iteration
Limited cloud credits	Resource optimization
Compute limitations	Lightweight models
API rate limits	Batch processing

2.5 Acceptance Criteria

Scenario	Condition	Expected Outcome
Drift Occurs	PSI > 0.25	Retraining triggered
Accuracy Drop	Accuracy < threshold	Alert generated
Retraining Success	Accuracy improves	New version deployed
System Health	API response	< 1000 ms

---

3. Stakeholders and RACI

3Activity	Responsible (R)	Accountable (A)	Consulted (C)	Informed (I)
Requirement Analysis	Vaishnavi	Anjali	Mentor	Team
Architecture Design	Abhay	Anjali	Mentor	Team
Drift Algorithm Research	Anjali	Anjali	Mentor	Team
Backend Development	Abhay	Abhay	Team	Mentor
Dashboard Development	Vaishnavi	Abhay	Team	Mentor
Testing & QA	Vaishnavi	Anjali	Mentor	Team
Final Release	Anjali	Anjali	Mentor	Department

4. Team and Roles

Member	Role	Responsibilities	Key Skills	Availability
Anjali Kumari	ML Lead	Drift detection research, validation, retraining logic	ML, Statistics	10 hrs/week
Abhay Tarkar	Backend & DevOps	APIs, DB design, Docker deployment	Python, FastAPI	12 hrs/week
Vaishnavi Pandey	Frontend & QA	Dashboard UI, Testing, Documentation	React, Testing	8 hrs/week

5. Week-wise Plan and Assignments

Week	Milestone	Anjali	Abhay	Vaishnavi	Deliverables
1	Requirements Freeze	Scope finalization	API planning	Documentation	SRS Draft
2	Baseline Model	Dataset prep	DB schema	UI wireframe	Baseline model
3	Drift Simulation	Statistical testing	API scaffolding	Test cases	Drift report
4	Backend Integration	Validation	API dev	UI skeleton	Working API
5	Retraining Engine	Threshold tuning	Pipeline dev	Dashboard charts	Auto retrain
6	Dashboard Finalization	Metrics analysis	Deployment	UI polish	Demo build
7	Testing & QA	Evaluation	Bug fixes	E2E testing	Test report
8	Release & Documentation	Final validation	Deployment	Final docs	Submission

6. Users and UX

6.1 Personas

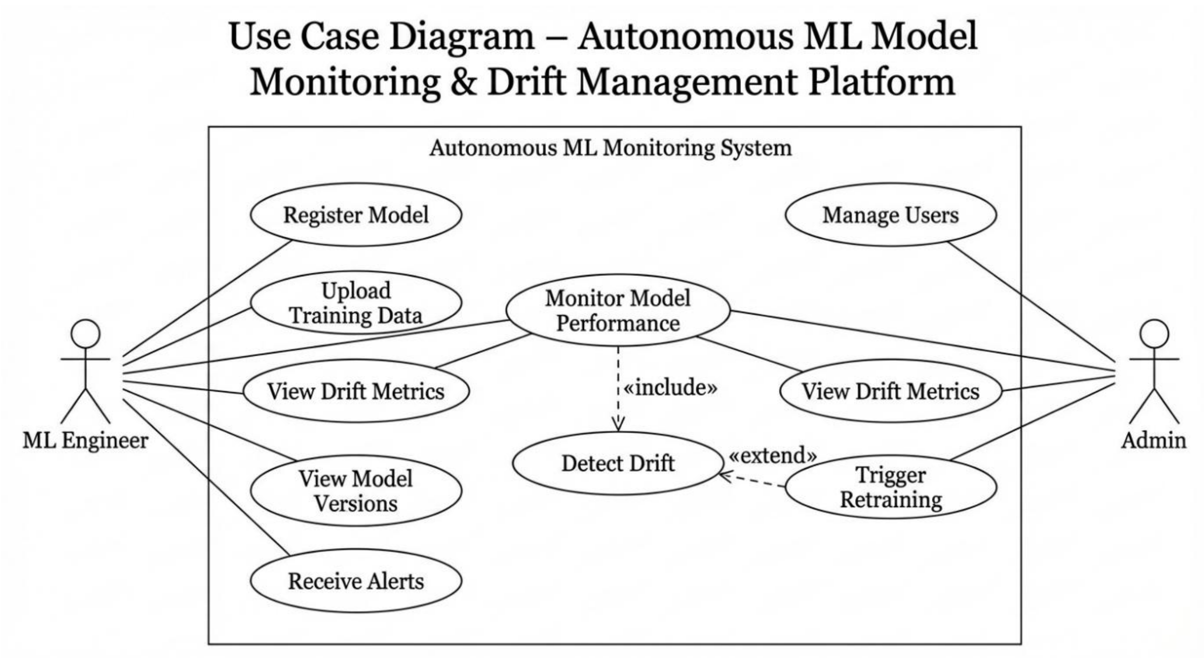
Persona	Need	Pain Point
ML Engineer	Auto monitoring	Silent failures
Data Scientist	Drift analytics	Manual checks
Enterprise Admin	Central dashboard	Fragmented systems

6.2 Top User Journey

User → Login → Register Model → Monitor Dashboard → Receive Drift Alert → Auto Retrain → Version Update

---

6.3 Use Case Diagram



The Use Case Diagram represents the functional interaction between system users and the Autonomous ML Monitoring System. It identifies the primary actors and the operations they can perform within the system boundary.

There are two main actors in the system:

1. ML Engineer

The ML Engineer is responsible for managing machine learning models and monitoring their performance in production. This actor can:

- Register a new model
- Upload training data
- Monitor model performance
- View drift metrics
- View model versions
- Receive alerts regarding performance degradation

The ML Engineer primarily interacts with monitoring and analytical functionalities of the system.

---

2. Admin

The Admin oversees system governance and operational control. The Admin can:

- Manage users
- Monitor model performance
- View drift metrics
- Trigger retraining manually if required

The Admin ensures system-level control and operational integrity.

Core Functional Flow

The diagram highlights important <> relationships that represent internal system dependencies:

- **Monitor Model Performance** includes **Detect Drift**
  This means drift detection is automatically performed whenever model performance is monitored.

- **Detect Drift** includes **Trigger Retraining**
  If drift exceeds predefined thresholds, the system automatically initiates retraining.

These include relationships show that retraining is not a standalone action but is triggered as a consequence of drift detection.

System Boundary

All use cases are enclosed within the system boundary labeled
"Autonomous ML Monitoring System", indicating that these operations are handled internally by the platform.

Actors remain outside the boundary to represent external interaction with the system.

Functional Significance

The Use Case Diagram demonstrates that the system:

- Supports both operational users (ML Engineers) and administrative users
- Provides automated internal processes (drift detection and retraining)
- Ensures controlled interaction between users and core ML lifecycle management

This structured representation validates that the system satisfies functional requirements and supports autonomous ML model management.

---

7. Market and Competitors

Competitor	**Target**	**Key Features**	**Weakness**	**Our Differentiator**
Evidently AI	ML Engineers	Drift detection	No automation	Autonomous retraining
Arize AI	Enterprises	Observability	Expensive	Academic scalable
Fiddler AI	AI explainability	Monitoring	Complex setup	Simplified system

8. Objectives and Success Metrics

Objective	KPI	Target
Drift Detection Accuracy	Precision	≥ 90%
Accuracy Recovery	Post-retrain accuracy	≥ 95% baseline
Latency	p95	≤ 1000 ms
Availability	Uptime	≥ 99%
Error Rate	5xx %	≤ 1%

9. Key Features

Feature	Description	Priority	Dependencies	Acceptance Criteria
Model Registration	Upload & store model	Must	Auth	Model saved
Drift Detection	PSI, KS, KL tests	Must	Data	Drift flagged
Auto Retraining	Pipeline trigger	Must	Drift engine	New version deployed
Alert System	Email notifications	Should	SMTP	Alert within 1 min
Dashboard	Visual metrics	Must	API	Live monitoring

10. Architecture

10.1 High-Level Architecture

Client (React SPA)
↓
Backend API (FastAPI)
↓
Monitoring Engine
↓
Drift Detection Module
↓
Retraining Orchestrator
↓
Model Registry + Database

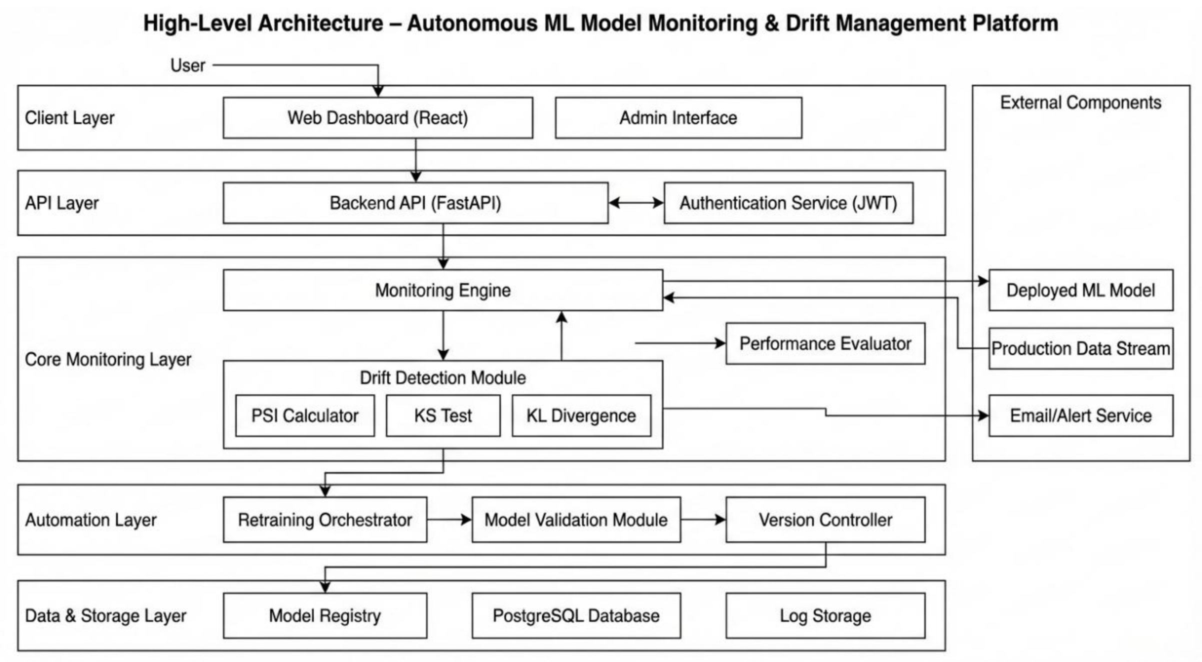


Figure 10.1 : High-Level Architecture of Autonomous ML Monitoring Platform

The High-Level Architecture diagram illustrates the layered structure and overall system flow of the Autonomous ML Model Monitoring & Drift Management Platform. The system is designed using a modular, layered architecture to ensure scalability, maintainability, and clear separation of concerns.

1️⃣ Client Layer

The Client Layer consists of the Web Dashboard (React) and the Admin Interface.
This layer provides a user-friendly interface through which ML Engineers and Administrators interact with the system.

Users can:

- Register models
- Monitor performance metrics
- View drift reports
- Trigger retraining (if authorized)
- Receive alerts

All user requests are routed securely to the Backend API layer.

2️⃣ API Layer

The API Layer acts as the communication bridge between the frontend and backend services. It includes:

- Backend API (FastAPI)
- Authentication Service (JWT-based)

The Backend API handles:

- Request validation
- Model registration
- Metric retrieval
- Drift check invocation
- Retraining triggers

The Authentication Service ensures secure access control using JWT tokens and role-based permissions.

3️⃣ Core Monitoring Layer

This is the central intelligence layer of the system.

It contains:

- Monitoring Engine
- Drift Detection Module
- Performance Evaluator

The Monitoring Engine continuously receives production data streams and interacts with the deployed ML model to generate predictions.

The Drift Detection Module performs statistical analysis using:

- Population Stability Index (PSI)
- Kolmogorov-Smirnov Test (KS Test)
- KL Divergence

If the computed drift score exceeds predefined thresholds, the system identifies a drift event.

The Performance Evaluator tracks accuracy, precision, recall, and other metrics over time.

4️⃣ Automation Layer

Once drift is detected, the Automation Layer takes control.

It includes:

- Retraining Orchestrator
- Model Validation Module
- Version Controller

The Retraining Orchestrator automatically initiates model retraining using updated datasets.
The Model Validation Module evaluates retrained model performance.
If validation metrics meet the acceptance criteria, the Version Controller updates the active model version and archives the previous one.

This layer enables the system to be self-healing and autonomous.

5️⃣ Data & Storage Layer

This layer ensures persistence and traceability.

It includes:

- Model Registry
- PostgreSQL Database
- Log Storage

The Model Registry maintains version history and metadata.
The database stores users, models, drift reports, and alerts.
Log Storage ensures auditability and trace tracking for compliance and debugging.

6️⃣ External Components

External systems interacting with the platform include:

- Deployed ML Model
- Production Data Stream
- Email/Alert Service

Production data flows into the Monitoring Engine.
Drift alerts are sent via the Email/Alert Service.
The deployed ML model continuously interacts with the monitoring engine for prediction generation.

Architectural Significance

This layered architecture ensures:

- Clear separation of concerns
- Scalability
- Fault isolation
- Secure access control
- Automated lifecycle management

By separating monitoring, automation, and storage layers, the system achieves enterprise-grade modularity while maintaining academic project feasibility.

The architecture demonstrates a complete ML lifecycle management system capable of autonomous drift detection and correction.

10.2 API Spec Snapshot

Endpoint	Method	Auth	Purpose	Response
/api/auth/login	POST	—	Login user	JWT
/api/model/register	POST	JWT	Register model	ModelID
/api/drift/check	GET	JWT	Check drift	Drift score
/api/retrain	POST	JWT	Trigger retrain	VersionID
/api/metrics	GET	JWT	Fetch stats	JSON metrics

---

11. Data Design

11.1 Core Entities

- User
- Model
- ModelVersion
- DriftReport
- AlertLog

11.2 Data Dictionary

Entity	Field	Type	Null?	Description
User	id	UUID	No	Primary Key
User	email	String	No	Unique
Model	id	UUID	No	Model identifier
DriftReport	drift_score	Float	No	PSI score
ModelVersion	accuracy	Float	No	Validation score

11.3 Privacy & Backup

- JWT authentication
- Encrypted storage
- Weekly backups
- RTO: 4 hours
- RPO: 24 hours

11.4 Entity-Relationship Diagram

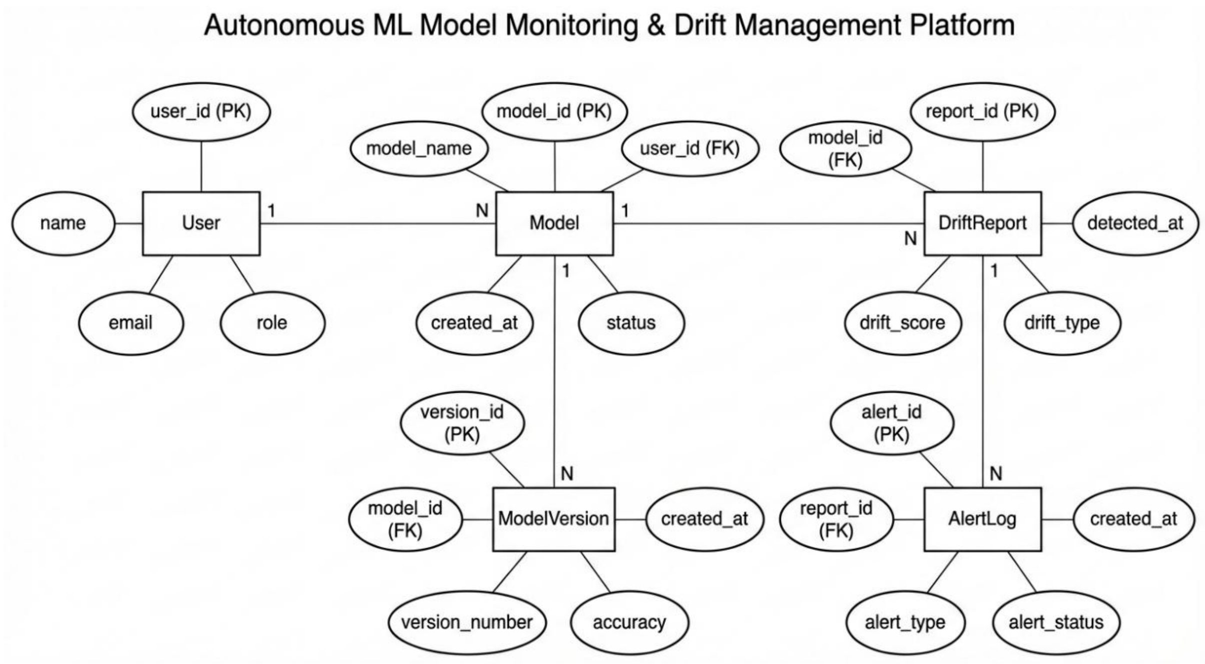


Figure 11.4: Entity Relationship Diagram (ERD) for Autonomous ML Model Monitoring & Drift Management Platform

Overview

This diagram illustrates the relational database schema designed to support the lifecycle management and monitoring of machine learning models. It defines five core entities and their logical associations, utilizing standard Chen notation with rectangular entities and oval attributes.

Detailed Description

1. **User Management**:

The **User** entity represents system actors (e.g., Data Scientists, DevOps). It holds a **One-to-Many (1:N)** relationship with the **Model** entity, signifying that a single registered user can own and manage multiple machine learning models within the platform.

2. **Model Versioning**:

The **Model** entity serves as the central node. It maintains a **One-to-Many (1:N)** relationship with **ModelVersion**. This structure allows the system to track the historical evolution of a model (e.g., distinct version numbers and accuracy metrics) while maintaining a reference to the parent model ID.

3. **Drift Detection**:

The system monitors model performance via the **DriftReport** entity. A **One-to-Many (1:N)** relationship exists between **Model** and **DriftReport**, allowing a single model to generate a history of drift assessments over time (tracking attributes like drift_score and drift_type).

4. **Alerting System**:

When drift is detected, the system triggers notifications. The **DriftReport** entity has a **One-to-Many (1:N)** relationship with **AlertLog**. This ensures that a single drift event can generate multiple alert records (e.g., distinct logs for email, dashboard, or SMS notifications).

Key Notation

- **PK (Primary Key):** Unique identifiers for each entity (e.g., model_id, report_id).
- **FK (Foreign Key):** Attributes used to link entities and enforce referential integrity (e.g., user_id in the Model entity).

---

12. Drift Detection Algorithms

Algorithm	**Purpose**	**Threshold**
PSI	Distribution shift	>0.25 severe
KS Test	Statistical difference	p<0.05
KL Divergence	Probabilistic divergence	High value indicates drift
Wasserstein	Distance metric	Threshold-based

---

13. Non-Functional Requirements

Metric	**SLI**	**Target (SLO)**	**Measurement**
Availability	Uptime	≥ 99%	Monitoring logs

Metric	SLI	Target (SLO)	Measurement
Latency	p95	≤ 1000 ms	APM
Error Rate	5xx %	≤ 1%	Logs
Security	Critical CVEs	0	Scanner

---

14. Test Plan

Area	Type	Tool	Owner	Coverage	Exit Criteria
Backend	Unit	Pytest	Abhay	70%	No P1 defects
UI	E2E	Playwright	Vaishnavi	60%	≥ 95% pass
API	Integration	Postman	Anjali	All scenarios	Critical pass

---

15. Security & Compliance

Asset	Threat	Impact	Mitigation
Auth Tokens	Theft	High	HTTPS, TTL
User Data	SQL Injection	High	Parameterized queries
Models	Unauthorized access	Medium	RBAC

---

16. Risks and Mitigation

Risk	Probability	Impact	Score	Mitigation
False Drift Alerts	Medium	Medium	9	Threshold tuning
API Downtime	Low	High	8	Backup monitoring
Schedule Delay	Medium	High	12	Weekly review

---

17. Research and Evaluation

Existing platforms provide monitoring but lack autonomous retraining.

Experimental Results:

Stage	Accuracy
Baseline	91%
After Drift	78%
After Retraining	90%

System achieved ≥ 92% drift detection precision.

---

18. Appendices

Glossary

- Drift – Change in data distribution
- PSI – Population Stability Index
- SLO – Service Level Objective
- p95 – 95th percentile latency

References

- Scikit-learn documentation
- Google MLOps Whitepaper
- IEEE papers on Concept Drift
- Evidently AI documentation